Test Markdown

Wuppie Fluppie

TL;DR

Text für TL;DR …

In Parr (2014) wird geschrieben, blablablabla …

Wir können hier sowohl Inline-Math ($a^i b^{2*i}$) als auch Block-Math:

$$\Phi(\mathbf{g}_i) = F(\Gamma(\mathbf{g}_i)) - w \cdot \sum_j (Z_j(\Gamma(\mathbf{g}_i)))^2$$

Code sollte auch gehen: `inline`, aber auch block:

```
fun fib(x) {
    if (x == 0) {
        return 0;
    } else {
        if (x == 1) {
            return 1;
        } else {
            fib(x - 1) + fib(x - 2);
        }
    }
}

var wuppie = fib(4);
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. In eu vulputate nisl. Ut scelerisque magna eros, at semper lectus vulputate vitae. Nullam egestas tellus lorem, eget sodales mauris lacinia ut. Etiam a viverra ex. Nam eu nisl vel nisl cursus condimentum. Aliquam accumsan augue ut consequat viverra. Curabitur sagittis est mauris, at molestie arcu condimentum ut. Quisque efficitur porta maximus. Donec non leo est. Aenean interdum condimentum libero, ac cursus dolor condimentum in. Nullam lorem ex, iaculis a orci vitae, iaculis aliquam enim.

Quisque est lacus, pellentesque vitae fringilla vitae, bibendum sit amet dolor. Proin rutrum metus sit amet hendrerit lobortis. Fusce ut ultrices enim. Morbi a urna rutrum, fringilla augue mattis, mollis lacus. Pellentesque elementum vitae magna ac feugiat. Vestibulum et metus eget augue finibus fringilla ac at velit. Cras eleifend in nisl ac commodo. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Suspendisse id mi nec diam pellentesque varius. Morbi in consequat neque, at bibendum purus. Quisque a est bibendum, pellentesque quam ac, fermentum diam. Cras augue nibh, tincidunt eu mollis at, hendrerit nec ex. Integer condimentum neque velit, eget aliquam justo iaculis ac.

Nam consequat vehicula faucibus. In consequat sed lacus sed congue. Ut quis risus vel erat tincidunt molestie. Nullam nibh lorem, placerat et dignissim porttitor, mollis eget nisl. Mauris eu justo nisi. Sed viverra, enim in tincidunt blandit, dui eros mattis nulla, in elementum neque ex sed diam. Aliquam nec neque vitae sapien pulvinar tempus. Donec gravida interdum nunc sed feugiat. Curabitur finibus sed urna at rutrum. Maecenas mollis pulvinar lobortis. Mauris dignissim orci ut metus eleifend, eget porttitor purus rhoncus. Phasellus volutpat egestas odio mollis pharetra. Cras sem dolor, commodo eu lectus vel, ultrices fringilla tellus. Fusce eleifend orci sed porttitor imperdiet. Phasellus suscipit, est ut semper blandit, ligula dui scelerisque felis, varius tincidunt tellus est vitae lectus. Phasellus eu molestie urna.

Sed sed leo vestibulum, iaculis justo in, aliquet mauris. Nunc luctus, metus quis vulputate lobortis, libero lacus imperdiet turpis, sit amet hendrerit magna nisi at leo. Pellentesque vehicula, mauris nec varius tempus, odio diam hendrerit mi, id porttitor ex felis ut sem. Vivamus tellus lacus, vehicula a augue non, aliquet aliquam quam. Donec dapibus quam sed ante blandit, in imperdiet enim dapibus. Etiam imperdiet sapien nec quam lobortis aliquet. Nam est eros, luctus consequat nulla ac, dapibus efficitur augue. Morbi tempor ex sed consectetur pharetra. Morbi ullamcorper ac enim nec accumsan. Nulla vestibulum eu turpis et rutrum.

Donec laoreet lectus at laoreet condimentum. Nulla porttitor elit iaculis turpis vestibulum, eget lobortis dolor pretium. Nulla sollicitudin convallis ultrices. Integer porttitor a nunc a viverra. Vestibulum pulvinar a urna et posuere. Duis sem nibh, consequat vitae ornare id, porttitor vel odio. Duis sit amet mi non odio accumsan condimentum sit amet non leo. Duis ut ligula ligula.

Phasellus sagittis non nisi eu ultricies. Donec quis ipsum at velit finibus mollis. Praesent facilisis blandit ligula. Pellentesque nec quam id neque lobortis egestas. Etiam nec risus feugiat, pulvinar mi in, vestibulum justo. Proin ac nisi scelerisque, bibendum tellus quis, lacinia leo. Vestibulum consequat fermentum est, eget cursus est efficitur ut. Fusce quis bibendum sapien. Vivamus vel nisl nulla. Nunc vehicula, odio ac ultricies iaculis, turpis turpis mollis dolor, et lobortis tortor ligula vel nisl. Nullam eget mi rutrum, suscipit felis at, dignissim justo. Praesent dapibus, arcu at luctus hendrerit, neque odio rhoncus lectus, vel semper lectus orci et ex. Etiam gravida, ex id hendrerit elementum,

---

GH-Preview

1 Hello World

Hier ist normaler Markdown-Text, mit **fett** und auch *kursiv*.

- Stichpunkt 1
- Stichpunkt 2
- Stichpunkt 3

1. Aufzählung 1
2. Aufzählung 2
3. Aufzählung 3
    1. Unterpunkt 3.1
    2. Unterpunkt 3.2

Hier die Pandoc-Markdown mark-Erweiterung.

2 Math

2.1 Inline

$\mathbf{g} = (g_1, \ldots, g_m) \in \{0,1\}^m$

- $a^i b^{2*i}$ ist nicht regulär
- $a^i b^{2*i}$ für $0 \leq i \leq 3$ ist regulär

2.2 Block

$$\Phi(\mathbf{g}_i) = F(\Gamma(\mathbf{g}_i)) - w \cdot \sum_j (Z_j(\Gamma(\mathbf{g}_i)))^2$$

$$p_{sel}(\mathbf{g}_k) = \frac{\Phi(\mathbf{g}_k)}{\sum_j \Phi(\mathbf{g}_j)}$$

$$g_i^{(t+1)} = \begin{cases} \neg g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ g_i^{(t)} & \text{sonst} \end{cases}$$

2.3 Known Problems

- VSCode Preview: `\mbox{ tanh }` => tanh => `\text{ tanh }` => tanh
- GH Preview:
  - `\phantom{xyz}` => => ?? => ??
  - `\operatorname{tanh}` => tanh => `\mathop{\text{tanh}}` => tanh

$$g_i^{(t+1)} = \begin{cases} \neg \quad g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ \quad g_i^{(t)} & \text{sonst} \end{cases}$$

Schwierig: In Pandoc-Markdown muss Mathe mit `$` oder `$$` eingeschlossen werden, unabhängig vom konkreten Inhalt. In LaTeX ist aber `\begin{eqnarray}` bereits der Beginn einer Mathe-Umgebung, d.h. hier wären extra `$$` falsch. Das muss per Filter korrigiert werden!

```
$$\begin{eqnarray}
S &\rightarrow& a A                          \nonumber \\
A &\rightarrow& d B \ | \ b A \ | \ c A  \nonumber \\
B &\rightarrow& a c \ | \ b C \ | \ c A  \nonumber \\
C &\rightarrow& \epsilon                     \nonumber
\end{eqnarray}$$
```

should become

$$\begin{eqnarray}
S &\rightarrow& a A \\
A &\rightarrow& d B \ | \ b A \ | \ c A \\
B &\rightarrow& a c \ | \ b C \ | \ c A \\
C &\rightarrow& \epsilon
\end{eqnarray}$$

3 Links

3.1 Link to WWW

craftinginterpreters.com/the-lox-language.html

3.2 Internal Links

selbe ebene: readme.md

unterordner: subfolder/foo.md

zurück nach oben I: ../02-parsing/antlr-parsing.md

zurück nach oben II: ../../homework/sheet01.md

4 Code

```
grammar Hello;

start : stmt* ;

stmt  : ID '=' expr ';' | expr ';' ;
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : ID | NUM ;

ID    : [a-z][a-zA-Z]* ;
NUM   : [0-9]+ ;
WS    : [ \t\n]+ -> skip ;
```

Java-Code kompilieren: `javac *.java`

Listing 1: The preprocessing step, cf. [@Dietz2018]

```
import org.antlr.v4.runtime.CharStreams;
import org.antlr.v4.runtime.CommonTokenStream;
import org.antlr.v4.runtime.tree.ParseTree;

public class Main {
    public static void main(String[] args) throws Exception {
        HelloLexer lexer = new HelloLexer(CharStreams.fromStream(System.in));
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        HelloParser parser = new HelloParser(tokens);

        ParseTree tree = parser.start();  // Start-Regel
        System.out.println(tree.toStringTree(parser));
    }
}
```

Code ohne alles

```
import org.antlr.v4.runtime.CharStreams;
import org.antlr.v4.runtime.CommonTokenStream;
import org.antlr.v4.runtime.tree.ParseTree;

public class Main {
    public static void main(String[] args) throws Exception {
        HelloLexer lexer = new HelloLexer(CharStreams.fromStream(System.in));
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        HelloParser parser = new HelloParser(tokens);

        ParseTree tree = parser.start();  // Start-Regel
        System.out.println(tree.toStringTree(parser));
    }
}
```

5 Images

Figures (w/ caption) should be centered like in LaTeX. Inline images will appear as is (also like in LaTeX).

5.1 Images with Caption

kleines Bild, keine Breiteangabe:

Abbildung 1: “B” (small)

---

kleines Bild, mit Titel und Breite:

Abbildung 2: “B”, width=“5%”

---

breites Bild, keine Breiteangabe:

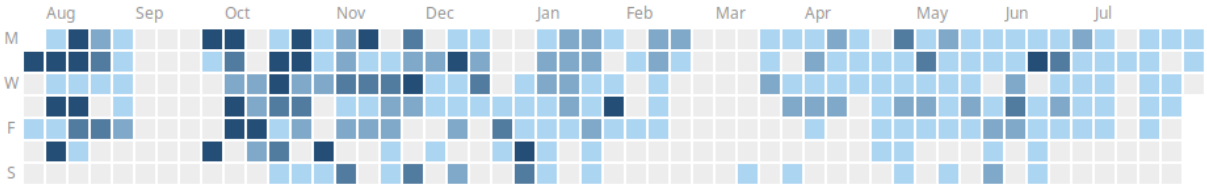


Abbildung 3: “wuppie” (wide)

---

breites Bild, mit Breiteangabe:

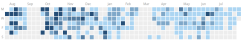


Abbildung 4: “wuppie”, width=“20%”

---

breites Bild über HTTP, keine Breiteangabe:

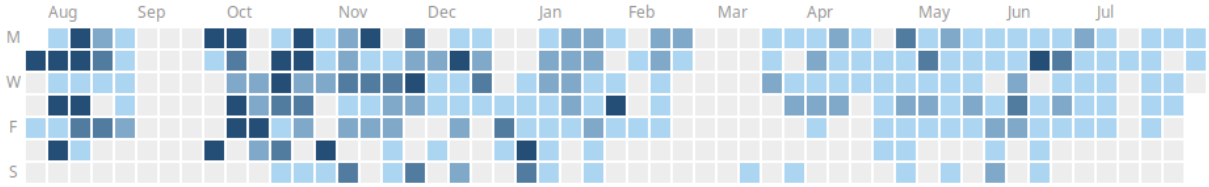


Abbildung 5: “wuppie” via web (raw)

---

breites Bild über HTTP mit `origin`-Span, keine Breiteangabe:

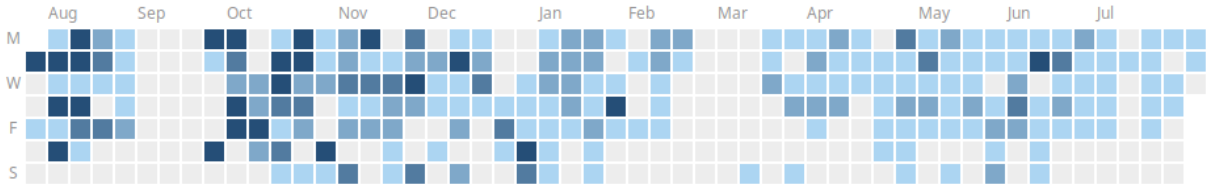


Abbildung 6: “wuppie” via web (raw) (Quelle: “FooFOOOO” by me on void.intern.com)

Quelle: “Foo” by me on void.extern.com

5.2 Images w/o Caption

ohne alles:

---

mit breitenangabe:

---

mit breitenangabe (“width”) und titel:

---

mit breitenangabe (“web_width”) und titel:

---

mit breitenangabe (“width” und “web_width”) und titel:

5.3 Images w/ `img`-Code

using “5%”:

using “80px”:

mit div drumherum:

5.4 Known Problems

- In VSC preview as well as in LaTeX images via web like https://github.com/cagix/pandoc-thesis/blob/master/figs/wuppie.png do not work (**need to be "raw"**)

6 Tabellen

mit caption:

Tabelle 1: Tabelle als Markdown-Pipe-Table, vgl. (Abelson, Sussmann, und Sussmann 1996)

Rechtsbündig	Linksbündig	Default	Zentriert
foo	foo	foo	foo
123	123	123	123
bar	bar	bar	bar

---

ohne caption:

Rechtsbündig	Linksbündig	Default	Zentriert
foo	foo	foo	foo
123	123	123	123
bar	bar	bar	bar

7 Zitieren, Quellen

Normales Zitieren (Siek 2023) …

Mit Seitenangabe (Siek 2023, 111) oder Kapitel (Siek 2023, Kap. 111) …

Als Author-Zitat Siek (2023) …

8 GFM

8.1 Details

Zusammenfassung: NIX :)

Lalelu …

8.2 Alert Extension

GH introduced “alerts” with distinctive styling, like

> [!NOTE] Foo bar, wuppie fluppie!

> [!TIP] Foo bar, wuppie fluppie!

> [!IMPORTANT] Foo bar, wuppie fluppie!

> [!WARNING] Foo bar, wuppie fluppie!

> [!CAUTION] Foo bar, wuppie fluppie!

(see https://github.blog/changelog/2023-12-14-new-markdown-extension-alerts-provide-distinctive-styling-for-significant-content/)

Let’s stick with Pandocs divs in Markdown content and use filters for export:

Foo bar, wuppie fluppie! Blablabla third line of nonsense …

Foo bar, wuppie fluppie!

Foo bar, wuppie fluppie!

Foo bar, wuppie fluppie!

Foo bar, wuppie fluppie!

- Export to GH Markdown using “distinctive alerts”
- Export to Hugo using notice shortcode
- Export to Beamer using beamercolorbox (also beameruserguide.pdf; or `block`, `alertblock`, `examples` - cf. https://www.overleaf.com/learn/latex/Beamer%23Creating_a_table_of_contents)

This should probably be in line with #180 …

9 Filter for Slides and Handouts

Foo bar, wuppie fluppie! (NOTES)

10 Footnotes

Sometimes[1] we need some[2] footnotes.

11 Handling of TeX Shenanigans

Zustand: (Formale) Beschreibung eines Zustandes der Welt

Aktion: (Formale) Beschreibung einer durch Agenten ausführbaren Aktion

- Anwendbar auf bestimmte Zustände
- Überführt Welt in neuen Zustand ("Nachfolge-Zustand")

LaTeX-Befehle wie `\bigskip` etc. sollten automatisch entfernt werden:

Hier nach den LaTeX-Befehlen.

Geeignete Abstraktionen wählen für Zustände und Aktionen!

12 Columns

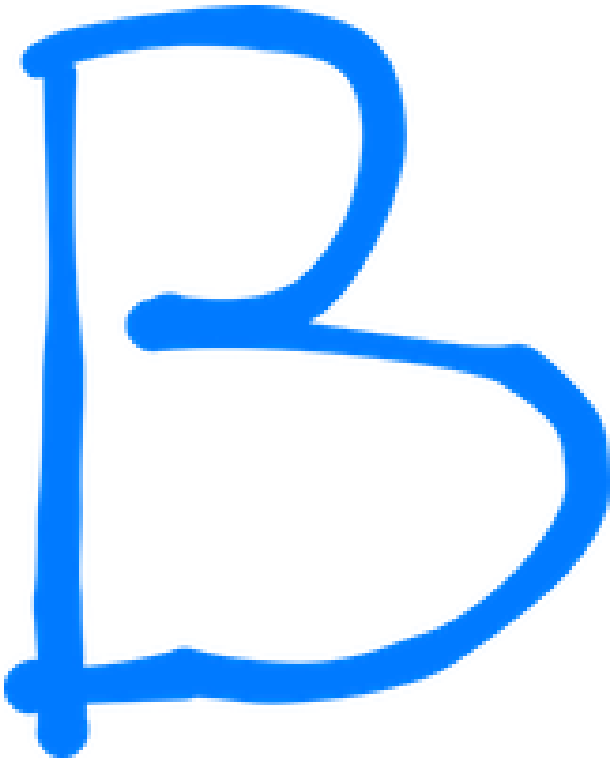
blablabla

Zum Parsen von Ausdrücken (*Expressions*) könnte man diese einfache Grammatik einsetzen. Ein Ausdruck ist dabei entweder ein einfacher Integer oder eine Addition oder Multiplikation zweier Ausdrücke.

```
expr : e1=expr '*' e2=expr      # MUL
     | e1=expr '+' e2=expr      # ADD
     | INT                      # NUM
     ;
```

Beim Parsen von "5*4+3" würde dabei der folgende Parsetree entstehen:

[1] sometime even more often
[2] lalalala



wuppie! fluppie! foo? bar …

13 Credits

Typische Regeln und Konventionen tauchen überall auf, beispielsweise bei Tim Pope (siehe nächstes Beispiel) oder bei "How to Write a Git Commit Message".

```
Short (50 chars or less) summary of changes

More detailed explanatory text, if necessary.  Wrap it to about
72 characters or so.  In some contexts, the first line is treated
as the subject of an email and the rest of the text as the body.
The blank line separating the summary from the body is critical
(unless you omit the body entirely); tools like rebase can get
confused if you run the two together.

Further paragraphs come after blank lines.

 - Bullet points are okay, too
 - Typically a hyphen or asterisk is used for the bullet, preceded
   by a single space, with blank lines in between, but conventions
   vary here
```

Quelle: "A Note About Git Commit Messages" by Tim Pope on tbaggery.com

14 Filters

14.1 ShowMe

Hier ein ShowMe-Test:

this is hidden content …

(but not used anymore)

Use `details` instead:

this is hidden content …

wuppie

this is a show-me w/ title :)

14.2 CBOX

this is content to be centered (and put into a box)…

(but not used anymore)

14.3 Center

this content should be centered

14.4 Alert

This will be highlighted. (but not used anymore)

Use Pandoc's mark extension instead: This will be highlighted. Even **with bold** text.

14.5 Hinweis

This is a hint. (but not used anymore)

14.6 Thema

The topic of this task or … (but not used anymore)

14.7 BSP

Lalalelu

Simple Beispiel-Button X (but not used anymore)

lalelu

use `ex` instead:

Simple Beispiel-Button X

Beispiel-Button w/ link

Vor `\pause`...

Nach `\pause`... (“neue” Slide)

15 Last Change

should be added automatically and in `\scriptsize` or `<sup>`<sub>

16 Zum Nachlesen

- Tate (2010, Kap. 2): foo bar wuppie fluppie
- Tate (2010, Kap. 2): Creating Graphical User Interfaces > Creating a GUI With Swing
- Nystrom (2021): Abschnitt 2.5.2: Ant
- (Nystrom 2021): Abschnitt 2.5.2: Ant

Lernziele

- k1: K1
- k2: K2
- k3: K3.1
- k3: K3.2

Quizzes

- Quiz Git Basics (ILIAS)

Challenges

Lexer und Parser mit ANTLR: Programmiersprache Lox

Betrachten Sie folgenden Code-Schnipsel in der Sprache "Lox":

```
fun fib(x) {
    if (x == 0) {
        return 0;
    } else {
        if (x == 1) {
            return 1;
        } else {
            fib(x - 1) + fib(x - 2);
        }
    }
}

var wuppie = fib(4);
```

Erstellen Sie für diese fiktive Sprache einen Lexer+Parser mit ANTLR. Implementieren Sie mit Hilfe des Parse-Trees und der Listener oder Visitoren einen einfachen Pretty-Printer.
(Die genauere Sprachdefinition finden Sie bei Bedarf unter craftinginterpreters.com/the-lox-language.html.)

Quellen

Abelson, H., G. J. Sussmann, und J. Sussmann. 1996. *Structure and Interpretation of Computer Programs*. MIT Press. https://mitpress.mit.edu/sites/default/files/sicp/index.html.

Nystrom, R. 2021. *Crafting Interpreters*. Genever Benning. https://github.com/munificent/craftinginterpreters.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/.

Siek, J. G. 2023. *Essentials of Compilation: An Incremental Approach in Racket*. The MIT Press. https://github.com/IUCompilerCourse/Essentials-of-Compilation.

Tate, B. A. 2010. *Seven Languages in Seven Weeks*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/seven-languages-in/9781680500059/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- “FooFOOOO” by me on void.intern.com
- “A Note About Git Commit Messages” by Tim Pope on tbaggery.com
- “Foo” by me on void.extern.com

> **Last modified:** 5f343d7 (add test for https://github.com/cagix/pandoc-lecture-zen/issues/32, 2025-05-14)